

Webserv – Coding Practices, SOLID & Design Patterns

This document is a pragmatic checklist for building a high-quality `webserv` (C++98/11 compatible) with strong engineering practices. It includes:

- Concrete do/don't rules
- SOLID principles with server-specific examples
- Design patterns mapped to subsystems
- Testing, security, performance, and CI habits
- Authoritative resources for each topic

1) Foundational C++ Practices

Core rules

- **Prefer RAII** for every file descriptor (sockets, pipes), directory handle, and process handle.
- **No raw `**/**`** in business logic. Use stack objects or dedicated owners (in C++98, manage with explicit dtors; in C++11, `unique_ptr` where allowed).
- **Fail fast at boundaries:** check every syscall return (`-1`) and set a consistent error path.
- **Never throw across the event loop.** Convert to status codes and log.
- **Const-correctness & immutability** wherever possible.
- **Zero global state;** pass dependencies explicitly (constructor injection).
- **Small headers, small translation units** → faster builds, clearer interfaces.
- **Strict warnings:** `-Wall -Wextra -Werror` (and `-Wpedantic` where possible). Enable ASan/UBSan in CI.

RAII snippet (C++98-friendly)

```
class unique_fd {
    int fd_;
public:
    explicit unique_fd(int fd = -1): fd_(fd) {}
    ~unique_fd(){ if (fd_ >= 0) ::close(fd_); }
    int get() const { return fd_; }
    int release(){ int t = fd_; fd_ = -1; return t; }
    void reset(int fd = -1){ if (fd_ >= 0) ::close(fd_); fd_ = fd; }
    // noncopyable
private: unique_fd(const unique_fd&); unique_fd& operator=(const unique_fd&);
};
```

String handling

- Avoid accidental copies; pass `const std::string&`.
- Normalize header keys to lowercase when storing; preserve original when needed.

Dependencies

- Keep third-party code out of critical paths. Vendor small utilities (mime types, tiny ini/lexers) if allowed.

Resources

- C++ Core Guidelines: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
- Google C++ Style Guide: <https://google.github.io/styleguide/cppguide.html>

2) SOLID Principles (with Webserv examples)

S — Single Responsibility

- `Connection` only drives the IO state machine; **does not** do routing or filesystem work.
- `StaticFileHandler` only maps URL→file and streams bytes.

O — Open/Closed

- Add new handlers (e.g., `RangeRequestHandler`) by implementing `IHandler` without modifying router core.

L — Liskov Substitution

- Any `IHandler` (`StaticFileHandler`, `CgiHandler`, `UploadHandler`) must be swappable without breaking `Connection`.

I — Interface Segregation

- Split interfaces: `IBodyReader`, `IResponder`, `ICgiEnvironment` rather than one fat `IServer`.

D — Dependency Inversion

- `Router` depends on abstractions (`IVirtualServerView`) not concrete storage. Inject `Logger&`, `Config&` via ctor.

Acceptance checks

- Swapping one handler for another must not change `Connection` logic or tests other than expectations.

Resources

- SOLID digest (short + practical): <https://refactoring.guru/solid>
-

3) Design Patterns Mapped to Webserv

Creational

- **Factory Method:** `HandlerFactory::create(location, request)` → returns `IHandler*` (static, upload, cgi...).
- **Builder:** `HttpResponseBuilder` builds headers/body progressively (status, headers, body or chunked).

Structural

- **Adapter:** wrap OS fds into `unique_fd` / `Pipe` classes; adapt `pollfd` to `PollerEvent`.
- **Composite:** `ConfigModel` (servers contain locations; locations contain directives).
- **Facade:** `ServerApp` orchestrates `EventLoop`, `Router`, `Logger`, `Config` simple entry API.
- **Decorator:** response filters (gzip, metrics) layered around `IResponder` (bonus).

Behavioral

- **State:** `Connection` state machine (READ_HEADERS → READ_BODY → DISPATCH → WRITE_HEADERS → WRITE_BODY → IDLE/CLOSE).
- **Strategy:** body parsing strategies (`ContentLengthParser`, `ChunkedParser`).
- **Chain of Responsibility:** routing pipeline: `ServerMatch` → `LocationMatch` → `MethodGuard` → `Handler`.
- **Command:** timer wheel callbacks encapsulated as commands.
- **Observer:** logging/metrics subscribers on connection/request lifecycle (optional).
- **Template Method:** base `AbstractHandler::handle()` with overridable hooks (`prepare()`, `produce()`).

Resources

- Design patterns (clear, code-oriented): <https://refactoring.guru/design-patterns>
-

4) HTTP Layer Practices

- **Robust parsing:** incremental line reader, enforce CRLF, header size cap, max header count.
- **Body handling:** `Content-Length` and (bonus) `chunked`. Reject both set simultaneously.
- **Keep-Alive:** honor `Connection: close/keep-alive`, server-side idle timeout.
- **Correctness:** Always set `Date`, `Server`, `Content-Length` (when known). Use default error pages.
- **Method enum:** `enum class HttpMethod { GET, POST, DELETE, UNKNOWN }`.

Resources

- HTTP/1.1 semantics: RFC 9110 <https://www.rfc-editor.org/rfc/rfc9110>
 - HTTP/1.1 messaging: RFC 9112 <https://www.rfc-editor.org/rfc/rfc9112>
 - MDN HTTP overview: <https://developer.mozilla.org/en-US/docs/Web/HTTP>
-

5) Routing & Config Practices

- **Deterministic matching:** select virtual server by `Host` + port; then longest-prefix location; apply deny/allow methods.
- **Data-driven:** no hardcoded roots; all from config.
- **Sanitize paths:** resolve canonical path and ensure it stays within location root.
- **Helpful config errors:** include source line/column in diagnostics.

Resources

- Nginx request processing (concepts): https://nginx.org/en/docs/http/request_processing.html
 - Nginx syntax reference (inspiration): <https://nginx.org/en/docs/syntax.html>
-

6) CGI Practices

- **Use ``;** never run interpreters in-process.
- **Env contract:** set `GATEWAY_INTERFACE=CGI/1.1`, `REQUEST_METHOD`, `SCRIPT_NAME`, `QUERY_STRING`, `CONTENT_TYPE`, `CONTENT_LENGTH`, `SERVER_PROTOCOL`, `SERVER_NAME`, `SERVER_PORT`, etc.
- **Timeouts:** hard kill runaway scripts; close pipes; reap children; propagate 504/502.
- **Security:** never pass unsanitized paths; drop privileges/chroot if allowed (bonus).

Resources

- CGI/1.1 spec (RFC 3875): <https://www.rfc-editor.org/rfc/rfc3875>
 - Beej's sockets guide (for pipes & fds too): <https://beej.us/guide/bgnet/>
-

7) Error Handling & Logging

- **Central error mapper** → HTTP status + message.
- **Access log:** common/combined format with request id.
- **Error log:** levels (DEBUG/INFO/WARN/ERROR), include errno text.
- **Never crash on user input;** respond with 4xx/5xx and continue serving others.

Resources

- Logging patterns & levels: <https://12factor.net/logs>

8) Security Practices

- **Header & request limits:** max header size, max headers, max body size per location.
- **Slowloris mitigation:** per-phase read deadlines; drop incomplete headers.
- **Path traversal:** canonicalize and check prefix against root; forbid symlink escapes.
- **CGI isolation:** env whitelist; no inherited sensitive env; time/memory/output caps.
- **Sensitive headers:** don't reflect raw `Host` / `Referer` without validation.

Resources

- OWASP Cheat Sheet Series: <https://cheatsheetseries.owasp.org/>

9) Performance Practices

- **Non-blocking IO** everywhere; `poll()` baseline; (bonus) `epoll/kqueue`.
- **Back-pressure:** output buffer watermarks; pause read when write backlog high.
- **Sendfile** (portable fallback) for large static files if allowed.
- **Pooling:** reuse small buffers; avoid per-request heap churn.

Resources

- man7 (`epoll`, `poll`, `sendfile`): <https://man7.org/linux/man-pages/>

10) Testing & TDD

Unit

- Config lexer/parser → AST → model (valid/invalid).
- Router precedence matrix.
- HTTP parser (headers, CL, errors, size caps).
- Multipart boundary parser for uploads.
- CGI env builder.

Integration/E2E

- Static: 200/404/403; redirects.
- Keep-alive reuse; idle timeout.
- Uploads: allowed/oversize.
- DELETE: success/permission/not found.
- CGI: good script, timeout, big output, malformed headers (→502).

Tooling

- Sanitizers (ASan/UBSan) on CI.
- Valgrind leak checks in nightly.
- Coverage thresholds on risky modules (parser/router $\geq 60\%$).

Resources

- Catch2 (tests): <https://github.com/catchorg/Catch2>
 - GoogleTest (alternative): <https://github.com/google/googletest>
-

11) Project Structure & Modularity

```
/src
  /core      (EventLoop, Poller, Timer, Sockets)
  /http      (Parser, Message, Router, Handlers)
  /server    (VirtualServer, Location, Limits)
  /cgi       (CgiHandler, CgiProcess, Env)
  /config    (Lexer, Parser, AST, Validator, Model)
  /util      (Logger, MimeTypes, Paths, Strings)
/include
/tests/{unit,integration}/fixtures
/assets/error_pages
```

Guidelines

- One class per file; headers expose types, not implementation details.
 - Keep constructors light; heavy init via `init()` that returns status.
 - Prefer plain old data (POD) `struct` for config models.
-

12) Git, Reviews & CI

- **Small PRs** (≤ 300 LOC diffs), clear titles `[module] short verb`.
- **Require green CI** before review: build, tests, sanitizers, clang-format.
- **Auto-merge** label when ready; branch protection enforces quality gates.
- **Housekeeping**: delete merged branches; move issues to Done automatically.

Resources

- GitHub Actions (workflows): <https://docs.github.com/actions>
 - Conventional Commits (optional): <https://www.conventionalcommits.org/>
-

13) Pattern-to-Module Cheat Sheet

- **Connection FSM** → State, Strategy (body readers)
 - **Router** → Chain of Responsibility, Strategy (matchers)
 - **Handlers** → Factory Method, Template Method, Liskov via `IHandler`
 - **CGI** → Adapter (pipes/fds), Command (timer callbacks), Facade (`CgiHandler`)
 - **Config** → Builder (model), Composite (AST), Interpreter (mini grammar)
 - **Logging/Metrics** → Observer (optional)
-

14) Final Do/Don't

Do

- Cap everything: headers, body, timeouts, CGI runtime.
- Return precise status codes and include a minimal HTML body.
- Centralize path resolution and security checks.

Don't

- Don't block the event loop (no long file ops on the main loop).
 - Don't let exceptions escape to the loop.
 - Don't share mutable state between connections; no singletons.
-

Appendix: Handy Specs & Manuals

- RFC 9110 (HTTP Semantics): <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 9112 (HTTP/1.1): <https://www.rfc-editor.org/rfc/rfc9112>
- RFC 3875 (CGI/1.1): <https://www.rfc-editor.org/rfc/rfc3875>
- Beej's Guide (sockets): <https://beej.us/guide/bgnet/>
- man7 (Linux syscalls): <https://man7.org/linux/man-pages/>
- Refactoring.Guru (patterns): <https://refactoring.guru/design-patterns>
- C++ Core Guidelines: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
- Google C++ Style Guide: <https://google.github.io/styleguide/cppguide.html>
- OWASP Cheat Sheets: <https://cheatsheetseries.owasp.org/>